2025/2026

FELCOM: A Secure Handheld Communication Device

Mini-Project

Prepared By:
Marc Gedeon
Yorgo Hassabou
Charbel Assaad

Presented To:
Dr. Hadi Jerdek

Table of Contents

Table of Figures

Abstract

FELCOM is a handheld radio that provides secure, text and voice communication in the 2.4 GHz ISM band. Each unit is built around an ESP32 microcontroller driving an nRF24L01+ transceiver, an INMP441 I2S microphone, a DAC, amplifier and speaker audio chain, an OLED display and tactile controls, all integrated on a custom PCB. To resist narrowband jamming and lower the probability of interception, the link continuously hops across six channels using Frequency-Hopping Spread Spectrum (FHSS). The nodes stay aligned by a shared, timer-driven hopping schedule. A custom 32-byte packet format multiplexes text, voice and control traffic over the same radio and carries a compact forward-error-correction header. A layered FEC subsystem in the transceiver provides reliability: a CRC-16 detects corruption on every protected packet, an automatic-repeat-request (ARQ) scheme guarantees exact delivery of chat messages, and an XOR block code reconstructs lost real-time audio packets without retransmission. Voice is captured as 8 kHz 8-bit PCM. Chat text is obfuscated with a lightweight XOR cipher. A built-in bit-error-rate (BER) test mode and live FEC counters allow link quality to be measured and analysed. The result demonstrates that robust, jam-resistant, low-cost digital voice and messaging can be realized entirely on commodity hardware.

1 Introduction

1.1 Background and Motivation

Reliable communication is usually taken for granted because it rests on a large, fixed infrastructure: cell towers, base stations, the Internet backbone. That infrastructure is also a single point of failure. In a natural disaster, in a remote area, or in any situation where the network is congested, censored or deliberately disabled, the same devices that work everywhere suddenly work nowhere. This motivates off-grid communication: a direct radio link between two people that depends on no third party.

A direct radio link, however, is exposed. A fixed-frequency transmitter is easy to locate, easy to intercept and trivial to jam, since a single interfering source on the right frequency is enough to silence it. The classical answer to this problem is spread spectrum. By rapidly and unpredictably changing the carrier frequency, a frequency-hopping system spreads its energy across a wide band: a narrowband jammer can only spoil the few hops that happen to land on its frequency, and an eavesdropper who does not know the hopping sequence sees only brief, scattered bursts. The same principle underlies Bluetooth today.

1.2 The Project

FELCOM implements these ideas on inexpensive, off-the-shelf hardware. It lets two units exchange text messages and live voice directly over the 2.4 GHz band. The link uses FHSS for resilience, employing a custom packet protocol with forward error correction, and obfuscates text with a lightweight cipher. Two additional modes—a real-time multiplayer Pong game and an RF/BER test screen—validate low-latency bidirectional traffic and raw link quality, respectively.

1.3 Objectives

The project aims to:

- build a working two-node handheld system on an ESP32 + nRF24L01+ platform,
- implement FHSS with a synchronization mechanism,
- define a flexible packet protocol that multiplexes text, voice, and control traffic over one radio,
- add a layered forward-error-correction subsystem (detection, retransmission, and forward recovery) matched to each traffic type,
- provide a debug mode that measures packet loss and bit-error rate, and
- integrate everything onto a custom PCB.

1.4 Technology and Methodology

The system uses the Espressif ESP32 (dual-core, hardware timers, I2S, and DAC peripherals) paired with the Nordic nRF24L01+, a 2.4 GHz GFSK transceiver that performs modulation in hardware and exposes 125 selectable 1 MHz channels, enabling FHSS.

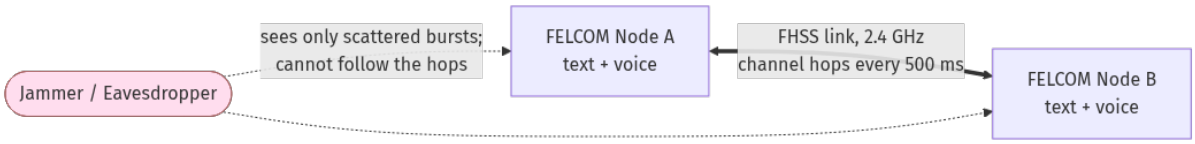


Figure 1: Two handhelds communicating directly over a frequency-hopping link

2 System Overview

2.1 Hardware

Each FELCOM unit is identical and runs the same firmware (only a one-byte `NODE_ID` differs).

The core is an ESP32 DevKit V1. Around it:

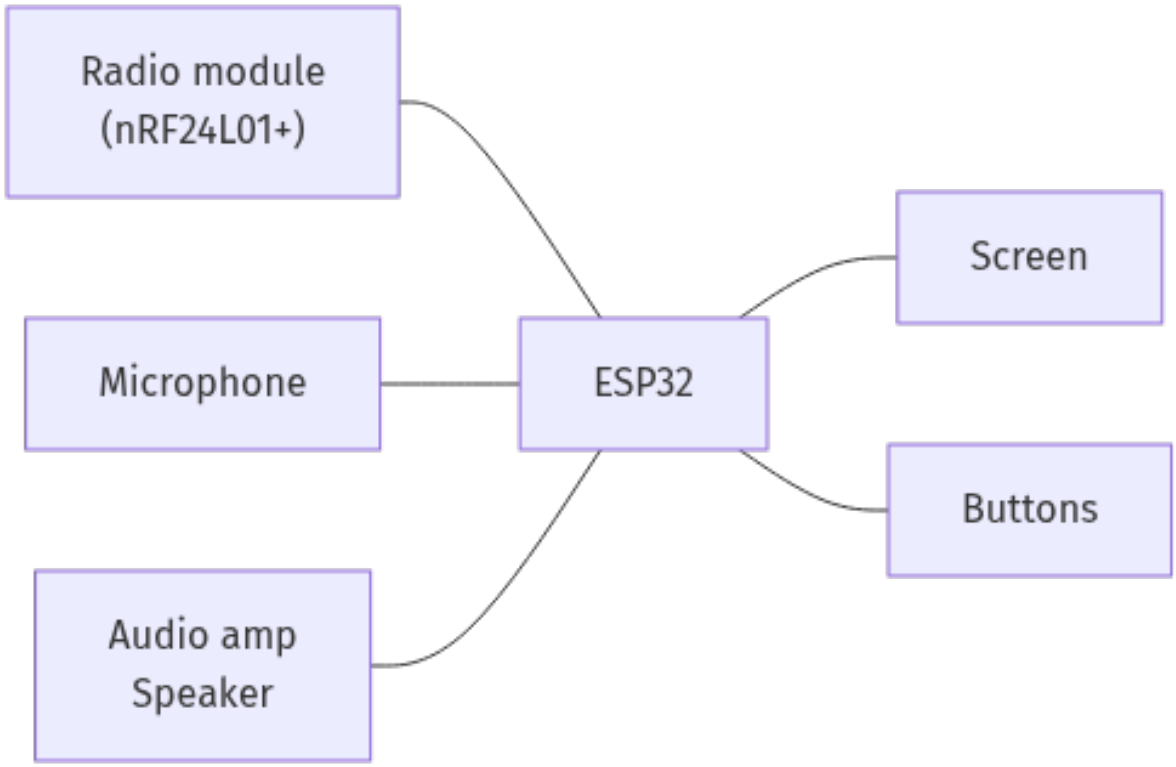


Figure 2: Block diagram of FELCOM

The complete schematic and routed PCB are shown below.

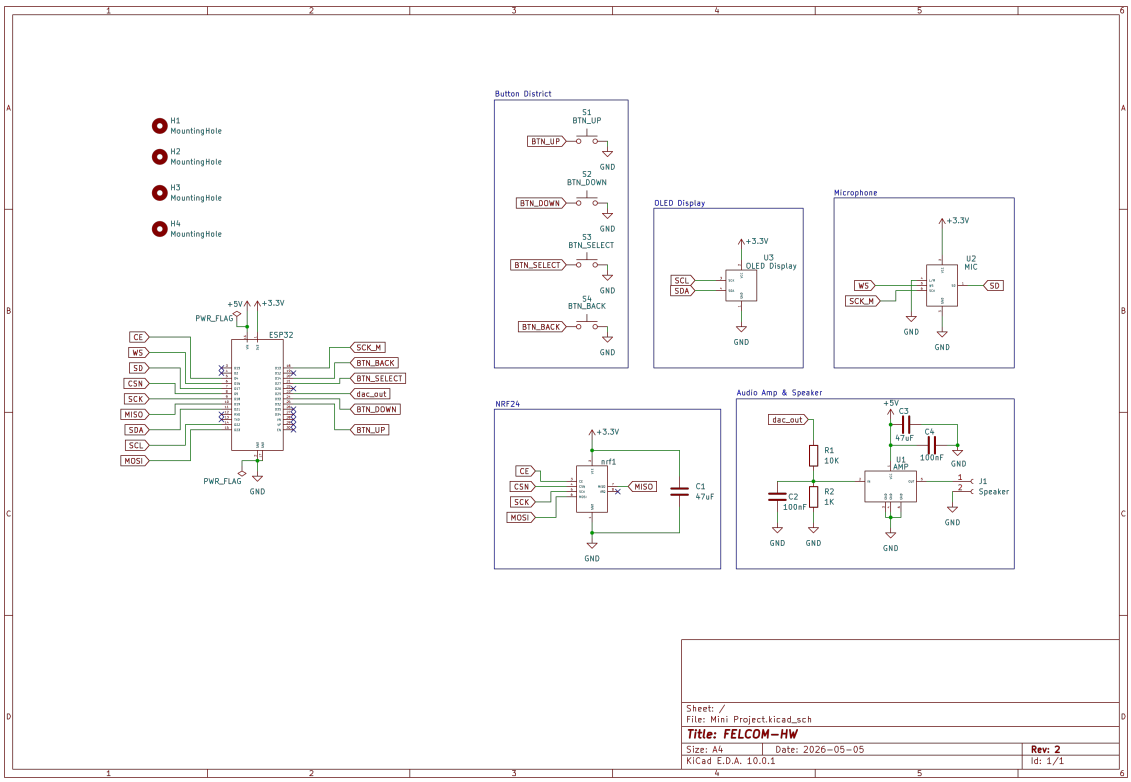


Figure 3: Full hardware schematic of a FELCOM unit (ESP32, nRF24L01+, INMP441, audio amplifier, OLED and controls)

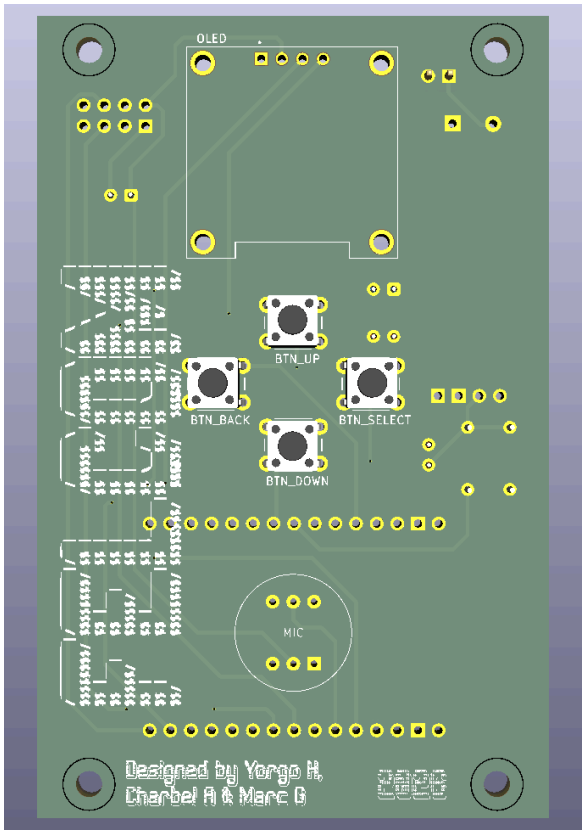


Figure 4: Custom-designed FELCOM PCB layout

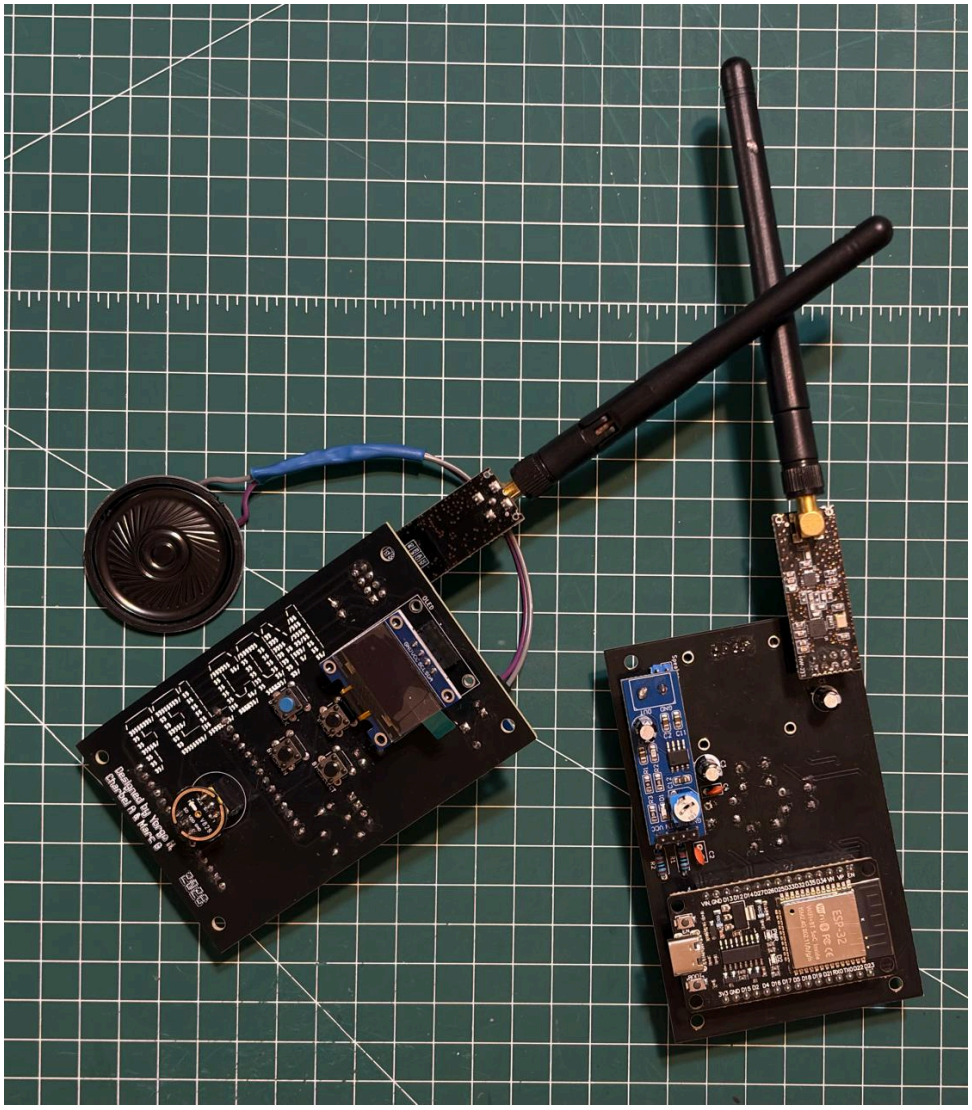


Figure 5: Two FELCOM units fully assembled

2.2 Firmware Architecture

The firmware is organized into clear layers, keeping each block (FHSS, framing, FEC, audio) independent and testable:

- **Physical layer**: the RF24 driver and the raw nRF24L01+ register access.
- **Transceiver layer** (`lib/transceiver`): performs channel access (CSMA), address filtering, and all error-control logic. The transceiver integrates the FEC subsystem, so application code never calls FEC routines directly, it simply uses `write`, `writeReliable`, `read`, `audioTx`, and `audioRx`.
- **Application layer**: chat, audio, Pong, Sync, and the RF/BER test, each with its own payload struct and OLED screen.

The main loop uses a cooperative, non-blocking design. Channel hopping, UI input, message handling, and audio capture/playback are all short steps pumped from a single `loop()`. A hardware timer increments a free-running counter every 500 ms, the loop reads it and hops when it changes. Since no step blocks, hopping always occurs on time, even while audio streams. This is the main constraint driving the architecture.

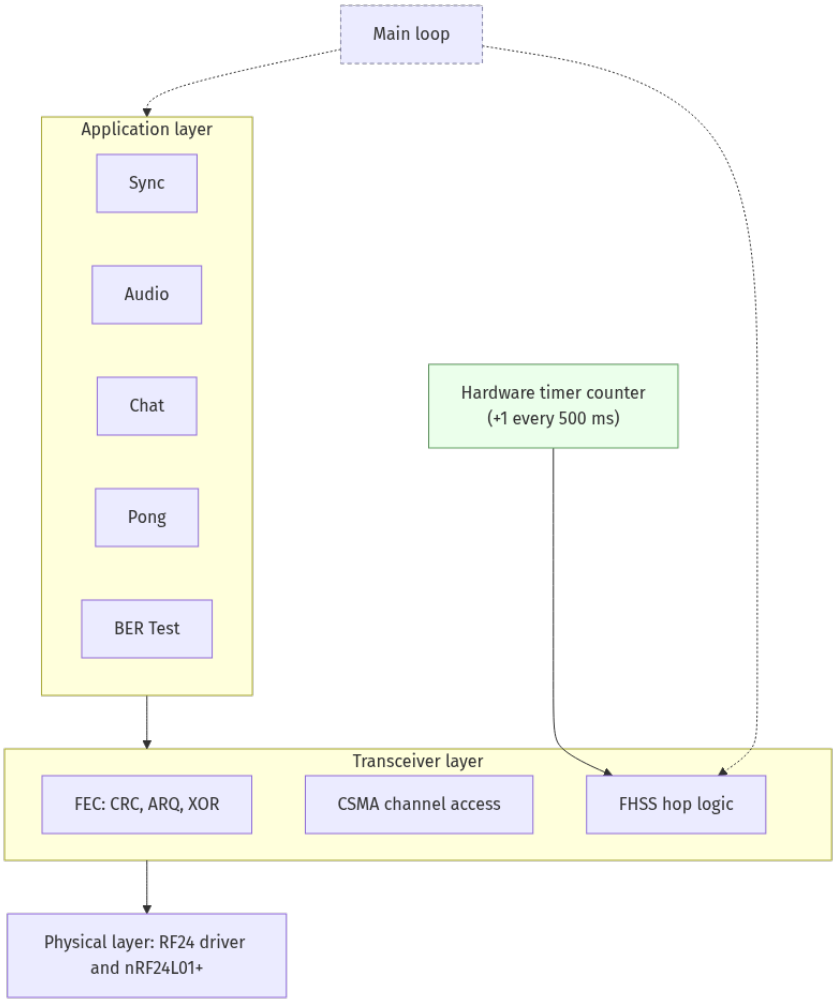


Figure 6: Layered firmware architecture and non-blocking main loop

3 Frequency-Hopping Spread Spectrum

FHSS is the heart of the project and provides its jam and detection resistance. The nRF24L01+ handles modulation in hardware while the firmware continuously changes the active channel from the transceiver's 125 available options.

3.1 Channel Selection

The 2.4 GHz band is crowded, primarily by Wi-Fi, whose channels are 20-22 MHz wide. Blind hopping would place many transmissions inside Wi-Fi channels, causing losses. The system therefore restricts hopping to the top of the band (approximately 2.51 GHz, corresponding to nRF24L01+ channels 110-115), which sits above standard Wi-Fi allocations and is typically quiet. The current build uses a deliberately small hop set of six channels, {110, 111, 112, 113, 114, 115}, trading some spreading gain for easier and faster synchronization.

3.2 Timer-Driven Hopping

All nodes share the same hop schedule. A hardware timer increments a global counter every 500 ms, the active channel is simply `HOPPING_CHANNELS[counter mod 6]`. Since all units derive the channel from the same counter value, they remain synchronized as long as their counters match. A full sweep through all six channels takes 3 seconds.

```
// Every 500 ms the shared timer counter ticks; the channel follows it.
uint32_t slot = readCounter() % HOPPING_CHANNELS_SIZE;
xcvr.setChannel(HOPPING_CHANNELS[slot]);   // one of 110..115
```

Listing 1: Timer-driven hopping: both nodes derive the channel from the same free-running counter, staying aligned without exchanging synchronization packets

3.3 Synchronization

Synchronization is critical for FHSS. The chosen approach uses timer-driven hopping with initial manual alignment.

A free-running hardware timer on each node increments a counter every 500 ms, and all nodes derive the current channel from this counter using `HOPPING_CHANNELS[counter mod 6]`. This ensures nodes remain synchronized as long as their counters match. Testing revealed that the hardware timers exhibit negligible drift: no measurable desynchronization occurred over 3 hours of continuous operation.

For initial alignment, a dedicated Sync screen allows users to manually synchronize two units before a chat or audio session. This one-time manual step initializes both counters to the same value, after which the hardware timer maintains alignment automatically.

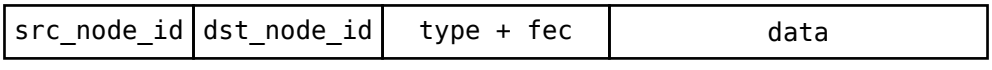
3.4 Channel Access (CSMA)

Since multiple packet types share the link, ordinary transmissions use Carrier Sense Multiple Access (CSMA): before transmitting, the radio briefly listens, if the channel is busy, it backs off for a random 1-10 ms interval and retries, up to five times. Real-time audio is the exception it deliberately bypasses CSMA backoff because millisecond level stalls would starve the audio pipeline.

4 The Packet Protocol and Data Link

Every transmission, regardless of type, is a fixed 32-byte frame, the maximum nRF24L01+ payload. A single frame format multiplexing all traffic is what lets text, voice and control share one hopping radio.

4.1 Packet Format



1 Byte 1 Byte 2 Bytes 28 Bytes

Figure 7: 32-byte packet format, 2-byte header + 28-byte payload

The 4-byte header contains two 1-byte address fields (`src_node_id` and `dst_node_id`) and a 2-byte bit-field packing `packet_type` (4 bits) with `fec` metadata (12 bits), leaving 28 bytes for payload. One-byte node IDs address packets, with `0xFF` reserved for broadcast. An RF test ping or sync beacon can thus reach all nodes simultaneously.

```
struct __attribute__((packed)) FHSSPacket {
    uint8_t  src_node_id;       // sender
    uint8_t  dst_node_id;       // 0xFF = broadcast
    uint16_t packet_type : 4;   // PONG / CHAT / AUDIO / ND_SYNC / TEST / ACK
    uint16_t fec : 12;          // scheme | XOR block id | slot index
    uint8_t  data[28];          // payload (CRC in the last 2 bytes)
};
```

Listing 2: C definition of the 32 byte packet: 4 byte header with bit packed fields + 28 byte payload

4.2 The Payload Region and the FEC Convention

A strict project-wide convention governs the 28-byte `data` region, ensuring the framing and error-control layers never overlap:

- For protected packets, the last 2 bytes of `data` hold a CRC-16, leaving 26 user bytes
- Reliable (ARQ) packets further use the first 2 of those 26 bytes for a sequence number, leaving 24 application bytes
- The 12-bit `fec` header field carries metadata only (scheme type, XOR block/slot identifiers), never the CRC itself, which would not fit in 12 bits

This single fixed layout allows one `read()` path to demultiplex all six packet types and deliver the correct payload to each application.

5 Forward Error Correction

The radio's built in CRC and auto-acknowledgement are disabled, error control is handled entirely by the custom FEC subsystem in the transceiver. This design choice allows tailoring the error control mechanism to each traffic type, as different applications require different guarantees. The subsystem combines three complementary mechanisms.

5.1 The Three Mechanisms

- **CRC-16-CCITT (detection)**: Every protected packet carries a 16-bit checksum in the last two payload bytes. On receipt, the CRC is recomputed, a mismatch indicates corruption and the packet is silently dropped. This transforms a noisy link into a clean-or-nothing channel.
- **ARQ (retransmission)**: For chat, which is infrequent but must be exact, the sender attaches a sequence number, transmits, and waits for an ACK, retransmitting on a 200 ms timeout up to three times. The receiver only ACKs after safely buffering the message, ensuring lost frames are re-sent rather than falsely confirmed. ARQ guarantees delivery but blocks briefly, so it is used only where latency is not critical.

- **XOR block code (forward recovery)**: For audio, which is real-time and cannot tolerate retransmission delays, the sender groups every four data packets and transmits a fifth parity packet equal to their bitwise XOR. If any one packet in a block is lost, the receiver reconstructs it by XOR-ing the four packets it did receive. Loss recovery operates in the forward direction only.

```
// TX: accumulate a running parity across the data slots of a block.
for (uint8_t b = 0; b < FEC_USER_SIZE; b++)
    parity[b] ^= pkt.data[b];

// After FEC_XOR_BLOCK_SIZE (=4) data packets, emit the parity packet.
if (++count >= FEC_XOR_BLOCK_SIZE)
    transmitAudioPacket(parityPkt);   // lets RX rebuild one lost packet
```

Listing 3: XOR forward error correction: parity packet is the bitwise XOR of four data packets, enabling reconstruction of any single lost packet

5.2 Per-Type Policy

Each packet type is matched to the mechanism that fits its needs:

Type	CRC	ARQ	XOR	Rationale
CHAT	✓	✓	✗	Infrequent; must be exact.
AUDIO	✓	✗	✓	Real-time; forward recovery only.
PONG	✓	✗	✗	Frequent; ARQ would backlog.
ACK	✓	✗	✗	Small control packet.
ND_SYNC	✗ (raw)	✗	✗	Must stay forgiving for resync.
TEST	✗ (raw)	✗	✗	Must measure real bit errors.

Table 1: Error Handling Strategy for Each Packet Type

TEST packets are sent completely raw so the BER screen measures the true channel error rate rather than a CRC-filtered version.

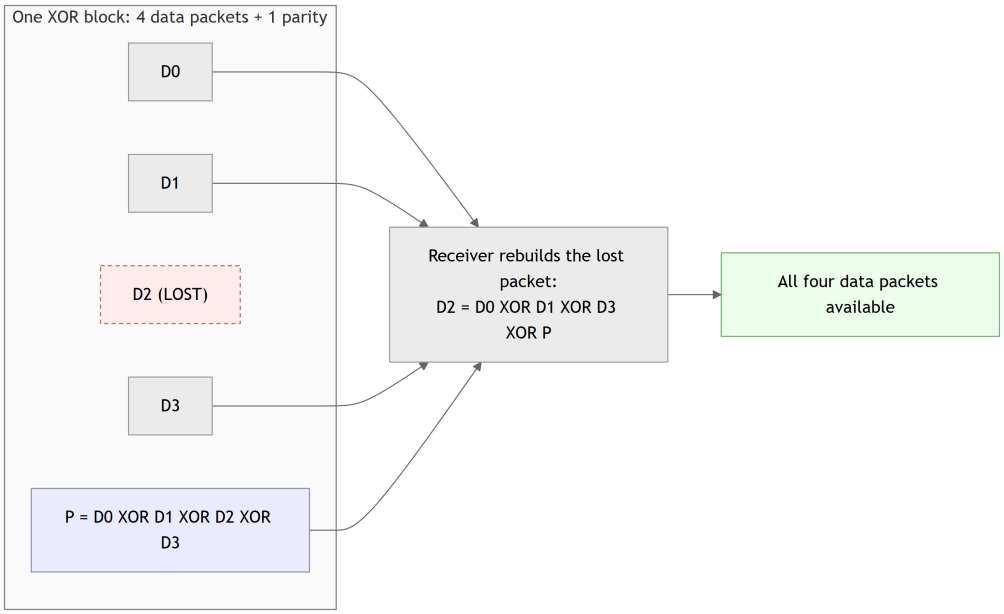


Figure 8: XOR block forward error correction packet reconstruction

6 Real-Time Audio

The audio mode turns the pair into a half-duplex walkie-talkie: one node talks, the other listens, and the Select button toggles the role. It is the most demanding feature because live voice must coexist with channel hopping that interrupts the link every 500 ms.

6.1 Format and Capture

Voice is captured from the INMP441 as 8 kHz, 8-bit mono PCM—deliberately low-fidelity but sufficient for intelligible speech while keeping the data rate low at 64 kbps. The microphone is read over I2S without blocking. Each sample is then conditioned in software: a running DC-offset estimate (exponential moving average) is subtracted, a gain factor is applied, and the result is clamped to 8 bits. Twenty-four conditioned samples (3 ms of audio) are packed into one `AudioPayload`, exactly filling the 26-byte protected user region.

6.2 Transport and Playback

Each payload is passed to `audioTx`, which wraps it with CRC + XOR-block FEC and transmits it, bypassing CSMA backoff. On the listening side, recovered payloads arrive through `audioRx` and are written into a large ring buffer that absorbs network jitter. Samples are then drained to the DAC at a precise 8 kHz using `micros()` timing. No additional hardware timer is needed because the hop counter already uses one.

6.3 Non-Blocking Pipeline

The entire chain capture, conditioning, transmission, reception, and playback—consists of short cooperative steps pumped from the main loop. Since nothing blocks, FHSS

hopping continues uninterrupted while audio flows. The XOR FEC is particularly important here: at a hop boundary, the in-flight packet is often lost, but the block code transparently rebuilds it, smoothing what would otherwise be an audible click every 500 ms.

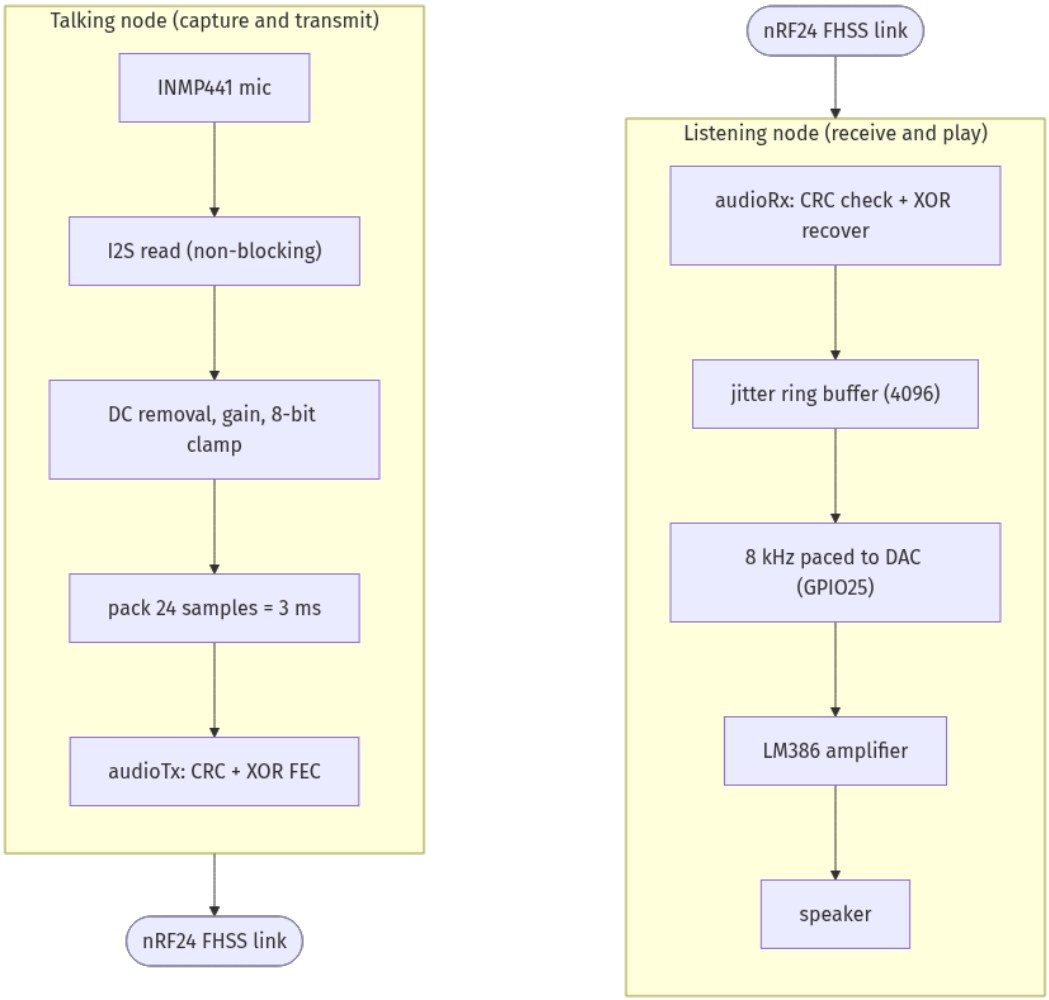


Figure 9: Non-blocking real-time audio pipeline from microphone to speaker

7 Security and Applications

7.1 Text Confidentiality

Chat text passes through a lightweight XOR stream cipher before transmission and is decrypted on receipt, protecting the message over the air. Combined with the unknown hopping sequence this provides a basic layer of confidentiality. It is important to note that this is obfuscation rather than strong cryptography. A stronger, authenticated cipher such as AES would be a clear security improvement.

7.2 Applications

Five screens exercise the stack:

- **Sync**: manually synchronizes two units by aligning their hop counters before a communication session
- **Chat**: encrypted text, sent point-to-point (reliable: ARQ + CRC) or broadcast (CRC only), displayed on a scrolling OLED log
- **Audio**: half-duplex walkie-talkie
- **Pong**: a two-player real-time game where paddle/ball updates validate low-latency bidirectional traffic over the hopping link
- **RF/BER Test**: a diagnostic mode that measures raw channel error rates

8 Testing, Results and Analysis

8.1 Method

Two diagnostic instruments are built into the firmware. First, the RF/BER test sends raw, known-pattern packets (repeated `0xAB`) at a fixed rate, the receiver echoes them back, and both sides count sent, received, echoed, and corrupt packets. Since TEST packets bypass FEC, this measures the true channel error rate. Second, the transceiver maintains live FEC counters (CRC pass/fail, ARQ sent/retransmitted/acked/failed, and audio blocks/recovered/lost), printed as a throttled serial summary. Together, these tools separate raw link quality from FEC recovery performance.

8.2 Results

To isolate the effect of frequency hopping, the RF/BER test was run twice on the same hardware and channel set: once with FHSS enabled, and once with hopping disabled so both nodes remained on a single fixed channel. Over 1000 transmitted packets in each configuration, the difference is clear:

Metric	**FHSS**	**No FHSS (fixed channel)**
Packets sent	1000	1000
Packets delivered	854 (85.4%)	480 (48.0%)
Packets lost	146 (14.6%)	520 (52.0%)
Bit-error rate (BER) (%)	1	19

Table 2: Link reliability with and without frequency hopping (RF/BER test, same hardware and channel set)

Beyond the raw channel, the three error-control layers were measured from the transceiver's live FEC counters during representative sessions: a text chat (which uses CRC + ARQ) and a real-time audio call (which uses CRC + XOR block recovery). The values below are cumulative totals across both nodes.

ARQ (reliable chat)	Value
Reliable messages sent	32
Delivered (ACK received)	29 (90.6%)
Failed after full retransmission	3 (9.4%)
Retransmissions triggered	9
Corrupt messages delivered (caught by CRC)	0

Table 3: ARQ reliability over a chat session (CRC + ARQ), both nodes combined

XOR + CRC (real-time audio)	Value
XOR blocks processed	9,255
Packets rebuilt by XOR parity	516
Packets lost (unrecoverable)	17,161
Corrupt packets dropped by CRC	2,027
Packets passing CRC	23,798

Table 4: Forward error correction over an audio session (CRC + XOR block code), both nodes combined

8.3 Analysis

Frequency hopping is the single biggest contributor to link reliability. On the fixed channel, more than half of the packets (52%) were lost and the bit-error rate reached 19%, because any persistent interferer on that one frequency corrupts every transmission. Spreading the same traffic across six hopping channels cut the loss to 14.6% and the BER to 1%, since only the few hops that coincide with an interferer are affected. Hopping turns a barely usable link into a usable one.

The error-control layers then act on top of the channel, and each behaves as designed. ARQ recovered transient losses through retransmission and delivered 29 of 32 chat messages; the three that failed did so only after exhausting their full retransmission budget, so they were reported as failed rather than silently dropped. CRC delivered zero corrupt chat messages, and during the much noisier audio session it detected and discarded 2,027 corrupted packets, keeping garbage out of the speaker.

The XOR block code shows both its value and its limit. It rebuilt 516 audio packets with no retransmission at all, which is exactly the forward recovery that real-time voice needs. However, 17,161 packets were lost beyond recovery: the parity packet can only repair one erasure per block of four, and the un-buffered audio link tends to drop several packets at each 500 ms hop boundary, so most damaged blocks lose more than one slot and cannot be reconstructed. This matches the audible glitching noted among the challenges, and points to a larger receive buffer or a stronger code as the clearest next improvement.

9 Challenges and How We Tackled Them

- **Synchronization**: The most significant challenge. After evaluating various approaches, we implemented timer-driven hopping with manual initial alignment. A dedicated Sync screen aligns counters before communication, after which the hardware timer maintains synchronization automatically.
- **Pong**: was initially running on an extremely low framerate due to having to process packets while rendering to the screen at the same time. The fix was simple, transmit packets less often since the most important aspect is the effect of the nodes on the ball position, not the paddle's position.
- **Audio pipeline**: initially too slow, causing signal overlap and transmission delays. The solution involved reworking the DAC buffer to drop old samples and bypassing CSMA backoff for audio packets.
- **CSMA stalling real-time audio**: The 1-10 ms CSMA backoff, suitable for chat, starved the audio pipeline and caused choppy playback. Audio transmission now bypasses backoff. Collisions are acceptable because the mode is half-duplex and FEC-protected.

10 Limitations and Future Work

- Synchronization requires manual initial alignment via the Sync screen. Automatic synchronization was considered but deferred due to time constraints.
- The nRF24L01+ modules proved unreliable. We had to test multiple units to find functional ones, and they occasionally drop packets.

11 Conclusion

FELCOM demonstrates that jam-resistant text and voice communication can be built on inexpensive hardware. Six-channel FHSS with timer-driven synchronization (manual initial alignment, then stable) provides the foundation, while layered FEC (CRC, ARQ, XOR) and a non-blocking design ensure reliability. Hardware limitations —notably nRF24L01+ inconsistencies and the lightweight XOR cipher—are acknowledged. Future work could add AES encryption and automated synchronization, turning this prototype into a resilient off-grid communication solution.

12 References

[1] Nordic Semiconductor, *nRF24L01+ Single Chip 2.4 GHz Transceiver Product Specification.*

[2] Espressif Systems, *ESP32 Technical Reference Manual* and *ESP32 Series Datasheet.*

[3] InvenSense (TDK), *INMP441 Omnidirectional Microphone with Bottom Port and I2S Digital Output, Datasheet.*

[4] Texas Instruments, *LM386 Low Voltage Audio Power Amplifier, Datasheet.*

[5] TMRh20, *RF24: Optimized Driver for nRF24L01(+) on Arduino & Raspberry Pi* (open-source library).

[6] H. K. Markey (Lamarr) and G. Antheil, *Secret Communication System*, U.S. Patent 2,292,387, 1942.

[7] ITU-T Recommendation V.41, *Code-independent error-control system* (CRC-16-CCITT).

[8] A. Goldsmith, *Wireless Communications*, Cambridge University Press (spread spectrum and FHSS fundamentals).